

Weight Tying & Sampling.

Two practical pillars of GPT-2. *Weight tying* reuses the input embedding matrix as the output projection, saving tens of millions of parameters. *Sampling* turns the final logits into generated text through temperature, top- k and top- p (nucleus) decoding. Every parameter count and every re-normalised probability below is computed in full — nothing summarised or deferred.

THE EQUATIONS

$$z = E^{\top} h \quad (\text{tied unembedding}), \quad p_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)} \quad (\text{temperature softmax})$$

Worked example. Tying saves $V \cdot d = 50257 \cdot 768 = 38,597,376$ parameters ($\approx 31\%$ of GPT-2 small). A 6-token logit vector decoded at $\tau = 0.7, 1.0, 1.5$, then filtered by top- $k=3$ and top- $p=0.9$.

Topic 3 of 3 in this module · companion to the course page · v1.0

Contents

1	Weight tying: one matrix, two jobs	2
2	Sampling: turning logits into tokens	3
3	Worked computation (real numbers)	3
4	Properties / consequences that matter	5
5	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 Weight tying: one matrix, two jobs

A decoder needs two vocabulary-sized matrices. The *input embedding* $E \in \mathbb{R}^{V \times d}$ maps each token id to a d -dimensional vector, $\text{embed}(x) = E_{x,:}$. The *output projection* (the “unembedding”) maps the final hidden state $h \in \mathbb{R}^d$ back to a logit per vocabulary entry, $z = Uh$ with $U \in \mathbb{R}^{V \times d}$. Weight tying sets $U = E$, so the *same* parameters do both jobs:

$$z = E h \in \mathbb{R}^V, \quad z_i = E_{i,:} \cdot h \quad (1)$$

Each logit z_i is the dot product of h with the embedding of token i : a token is predicted strongly exactly when h points in the direction of that token’s embedding. (Equivalently, if one stores the table as $E^\top \in \mathbb{R}^{d \times V}$, the projection is $z = (E^\top)^\top h$; the content is identical.)

The input map answers “what vector represents token i ?” and the output map answers “how much does this hidden state look like token i ?” These are the same geometric question asked in two directions, so it is natural — and empirically helpful — to use one shared matrix. Untying them forces the model to learn the same vocabulary geometry twice.

1.1 Parameters saved (worked)

An untied model stores E and U separately: $2Vd$ parameters. A tied model stores one: Vd . The saving is the size of a whole vocabulary matrix,

$$\Delta = Vd = 50,257 \times 768 = 38,597,376 \text{ parameters.}$$

GPT-2 small has $\approx 124.4\text{M}$ total parameters, so this single matrix is

$$\frac{38,597,376}{124,439,808} \approx 31.0\%$$

of the entire model — removed from the trainable count without losing capacity. The saving scales with V , so it is most dramatic for the large vocabularies of modern tokenizers.

Tying $U = E$ removes one $V \times d$ matrix from the model. For GPT-2 small ($V = 50257$, $d = 768$) that is 38,597,376 parameters — about 31% of all 124M — and it typically *improves* validation perplexity, because gradients from both the embedding lookup and the output head now refine a single, better-estimated table.

2 Sampling: turning logits into tokens

At generation time the model produces logits $z \in \mathbb{R}^V$ for the next token. We need a rule that turns z into an actual choice. All the standard rules reshape the distribution before sampling from it.

2.1 Temperature

A temperature $\tau > 0$ rescales the logits before the softmax:

$$p_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)} \quad (2)$$

$\tau < 1$ divides by a small number, magnifying logit gaps and *sharpening* the distribution toward the top token; $\tau > 1$ shrinks the gaps and *flattens* it toward uniform; $\tau \rightarrow 0^+$ makes the top logit dominate completely, recovering greedy arg max decoding.

2.2 Top- k

Keep only the k tokens with the largest logits, set the rest to probability 0, and renormalise over the survivors:

$$p_i^{(k)} = \frac{\mathbf{1}[i \in \mathcal{K}] \exp(z_i)}{\sum_{j \in \mathcal{K}} \exp(z_j)}, \quad \mathcal{K} = \text{indices of the } k \text{ largest logits.}$$

2.3 Top- p (nucleus)

Keep the *smallest* set of top tokens whose cumulative probability reaches p , then renormalise:

$$\mathcal{N}_p = \text{smallest prefix (by descending } p_i) \text{ with } \sum_{i \in \mathcal{N}_p} p_i \geq p.$$

Unlike top- k (a fixed count), the nucleus size *adapts*: it is small when the model is confident (one token already exceeds p) and large when the model is unsure (probability is spread out).

3 Worked computation (real numbers)

Take a 6-token logit vector

$$z = \begin{bmatrix} 3.0 & 2.0 & 1.0 & 0.5 & 0.0 & -1.0 \end{bmatrix}^\top \quad (\text{tokens } A, B, C, D, E, F).$$

3.1 Step A — temperature softmax at $\tau = 0.7, 1.0, 1.5$

Apply (2) for each τ (subtracting $\max(z/\tau)$ for stability). Results:

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>
$\tau = 0.7$	0.745011	0.178543	0.042788	0.020946	0.010254	0.002457
$\tau = 1.0$	0.604813	0.222498	0.081853	0.049646	0.030112	0.011078
$\tau = 1.5$	0.460679	0.236521	0.121434	0.087011	0.062346	0.032010

As τ rises the top token *A* loses mass ($0.745 \rightarrow 0.605 \rightarrow 0.461$) and the tail fills in: exactly the sharpening/flattening trade-off. The Shannon entropy $H = -\sum_i p_i \log p_i$ rises with τ : 0.8045, 1.1478, 1.4497 nats, a direct measure of how much more diverse (uncertain) the sampling becomes.

Temperature reshapes the distribution (rows τ ; intensity scales with p , capped at 80)

	A	B	C	D	E	F
$\tau=0.7$	0.745	0.179	0.043	0.021	0.010	0.002
$\tau=1.0$	0.605	0.222	0.082	0.050	0.030	0.011
$\tau=1.5$	0.461	0.237	0.121	0.087	0.062	0.032

3.2 Step B — top- $k = 3$ at $\tau = 1$

The three largest logits are *A, B, C* (logits 3, 2, 1); *D, E, F* are dropped. Renormalise over $\{A, B, C\}$ by exponentiating the surviving logits and dividing by their sum ($e^0 + e^{-1} + e^{-2} = 1.503$ after subtracting $\max = 3$):

$$p^{(k)} = (\underbrace{0.665241}_A, \underbrace{0.244728}_B, \underbrace{0.090031}_C, 0, 0, 0), \quad \sum = 1.$$

The kept mass (originally $0.605 + 0.222 + 0.082 = 0.909$) is rescaled up to 1.

3.3 Step C — top- $p = 0.9$ at $\tau = 1$

Sort the $\tau=1$ probabilities descending and accumulate:

token	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>
p_i (desc.)	0.6048	0.2225	0.0819	0.0496	0.0301	0.0111
cumulative	0.6048	0.8273	0.9092	0.9588	0.9889	1.0000

The cumulative sum first reaches 0.9 at the third token, so the nucleus is $\mathcal{N}_{0.9} = \{A, B, C\}$. Renormalise over the nucleus (divide by 0.909164):

$$p^{(p)} = (0.665241, 0.244728, 0.090031, 0, 0, 0).$$

Here top- $p=0.9$ and top- $k=3$ happen to select the same three tokens and give identical renormalised probabilities — but if *A* alone had exceeded 0.9, the nucleus would shrink to $\{A\}$, whereas top- k would still keep three. That adaptivity is the point of nucleus sampling.

4 Properties / consequences that matter

1. **Greedy is the cold limit.** $\tau \rightarrow 0^+$ concentrates all mass on $\arg \max_i z_i$: temperature sampling continuously interpolates between greedy ($\tau \rightarrow 0$) and uniform-ish (τ large).
2. **Filtering then renormalising preserves order.** Top- k and top- p never reorder the surviving tokens' relative probabilities; they only zero out a tail and rescale. They are usually combined with temperature (apply τ , then filter, then sample).
3. **Entropy as a diversity dial.** $H(p)$ grows with τ and shrinks under top- k /top- p ; it is a single scalar summarising how varied the next-token choice is, and (averaged, exponentiated) it is the perplexity of Topic 2.
4. **Tying is free capacity.** Removing Vd parameters via tying lowers memory and regularises the model; it is standard in GPT-2 and most successors.

Renormalise after truncating. Top- k and top- p zero out tokens, so the survivors no longer sum to 1. You must divide by the surviving mass before sampling — otherwise you are sampling from a sub-stochastic vector and the effective temperature drifts. Likewise, apply temperature *before* top- p : changing τ after computing the cumulative cutoff would invalidate the nucleus you just selected.

5 Where this sits in the track

The logits z decoded here are produced by the tied projection (1) from the hidden state of GPT-2's final layer; they are the same logits whose softmax defined the training loss in Topic 1 and the perplexity in Topic 2. Training optimises that distribution; sampling *consumes* it. The entropy used as a diversity measure is the per-step uncertainty whose averaged exponential is perplexity, closing the loop on this module.

A Reproducible (NumPy)

Inputs: $z = (3, 2, 1, 0.5, 0, -1)$; $V = 50257$, $d = 768$. Intermediates: tied saving $Vd = 38,597,376$; softmax at $\tau = 0.7, 1.0, 1.5$ as tabulated; entropies $(0.8045, 1.1478, 1.4497)$; top- $k=3$ and top- $p=0.9$ both give $(0.665241, 0.244728, 0.090031)$ over $\{A, B, C\}$.

```
import numpy as np
V, d = 50257, 768
print(V*d)                                # 38597376 params saved by tying

z = np.array([3.0, 2.0, 1.0, 0.5, 0.0, -1.0])
def softmax(x):
    x = x - x.max(); e = np.exp(x); return e / e.sum()
for tau in (0.7, 1.0, 1.5):
    p = softmax(z / tau)
    H = -(p * np.log(p)).sum()              # entropy in nats
    print(tau, p, H)
```

```
p = softmax(z)                                # tau = 1.0
k = 3; keep = np.argsort(z)[-k:]               # top-k indices
pk = np.zeros_like(p); pk[keep] = np.exp(z[keep]); pk /= pk.sum()

order = np.argsort(p)[::-1]                   # top-p = 0.9
cum = np.cumsum(p[order]); n = np.searchsorted(cum, 0.9) + 1
nuc = order[:n]; pp = np.zeros_like(p)
pp[nuc] = p[nuc] / p[nuc].sum()
print(pk, pp)                                # both -> 0.665241 0.244728 0.090031
```

Marevlo Research · Transformer Track · Module 2.1 · Topic 3 of 3.